

Verify, Then Delete: A Method for Collapsing a Kernel Cartesian Product

Hanzo AI — ML Systems

hanzo-kernel Paper 07.6 — part of the *One Source, Every Backend* series

Abstract

The other papers in this series build the machinery to write a GPU kernel once and lower it to every backend. This paper is about the migration: how you drain 52,345 lines of hand-written kernel arithmetic across five backend trees down toward a single source *without* a big-bang rewrite and without ever shipping a slower engine. The method is a discipline we call **verify-then-delete**: never retire a hand-tuned kernel on faith, and never all at once. A hand-written kernel is deleted only when its single-source twin is (a) bit-exact against the shared CPU oracle *and* (b) at least as fast on a kernel-only benchmark — in practice $\geq 97\%$ of the incumbent’s throughput. A per-(**op**, **type**, **backend**) registry flips one cell to the DSL the moment its twin passes both gates, so the product drains to a sum one measured cell at a time, with a working, faster-or-equal engine at every intermediate state. We give the ledger, the protocol, and the measured outcomes — including a migration that was *7.1× faster* than the shader it replaced and bit-exact, and, just as importantly, the migrations the gate correctly *declined*.

1 The ledger: what there is to collapse

Table 1 is the denominator. It is the hand-written kernel arithmetic in the Hanzo ML and engine repositories: the same ~ 20 operations, specialized over ~ 22 GGUF quantization formats, re-implemented once per backend across five trees and five shading languages.

Tree	Language	Lines
CUDA (ML)	CUDA C++	13,123
ROCm	HIP C++	6,725
Metal	Metal Shading Language	3,530
Vulkan	GLSL $\rightarrow$ SPIR-V	7,105
CUDA (engine)	CUDA C++	21,862
Total	5 trees	52,345

Table 1: The collapse ledger: hand-written, non-generated, non-vendored kernel arithmetic.² Roughly 85% is a deletable target — the duplicated arithmetic — as opposed to the genuinely peak, genuinely per-backend kernels (block-quant tensor-core MMQ, [6]) that stay behind islands.

²These are the *deletable* lines. A naïve `wc -l` over all `*.cu/*.cuh/*.hip/*.hpp/*.metal/*.comp` is roughly $2\times$ larger ($\sim 112k$) because it also counts C++ headers, a `target/package/` build-artifact copy, machine-generated codebook tables (the i-quant grids), and vendored upstream kernels (FlashInfer, Marlin, GPTQ) — none of which are hand-written duplication or a DSL collapse target.

The product is not merely a maintenance cost; it is the breeding ground for the divergence bugs the oracle paper catalogs ([5]). Draining it is therefore a correctness project as much as a code-size one. But it must be drained safely: a rewrite that regresses performance, or that lands as one enormous untestable change, is worse than the duplication it removes.

2 The protocol: verify, then delete

The method is three rules, applied per cell of the types $\times$ backends matrix.

1. **Coverage first.** The DSL fills the entire matrix from one source. Many cells were previously *missing* or buggy; those gain correct coverage immediately, before any deletion — a strict improvement with nothing to retire.
2. **The two-part gate.** A hand-tuned kernel is retired for its single-source twin *only* when the twin is both:
 - **bit-exact** against the shared CPU oracle (the correctness half; [5]), and
 - **at least as fast** on a kernel-only benchmark — buffers created once, N dispatches enqueued back-to-back, one final sync, so the number reflects the kernel and not the host round-trip. In practice the bar is $\geq 97\%$ of the incumbent’s throughput.

The number decides, per quant type, per backend. Not a human’s confidence — the benchmark.

3. **Delete on parity.** When a twin passes both gates, its hand-written copies are deleted and a per-(op, type, backend) registry flips that cell to the DSL, keyed by the gate result. Until it passes, the cell keeps its hand-tuned kernel. The registry is the seam that lets the product drain to a sum one measured cell at a time, with a working, faster-or-equal engine at every intermediate state.

The protocol is reversible and never a downgrade by construction. It is a scoreboard with a gate, not a rewrite.

3 Measured migrations: the wins

The method is not hypothetical; cells have flipped on **main**. All figures below are kernel-only benchmarks on the named hardware.³

Portability costs nothing (dp4a matvec). The central fear of any portable DSL is a performance downgrade. It does not materialize on the bandwidth-bound matvec that dominates decode. A tuned single-source **dp4a Q8 matvec** — **Vector<i8,4>→OpSDot**, one thread block per output row with threads striding the groups so adjacent threads read adjacent packed weights (coalesced 256-byte transactions), partials tree-reduced in shared memory — measures **196 GB/s** on evo (8192², cache-busting), which is **118% of the ~166 GB/s hand-tuned kernel** it replaces and $\sim 77\%$ of the DRAM roofline, bit-exact. The naïve one-thread-per-row version that started at ~ 17 GB/s was memory-*uncoalesced*, not DSL-limited; the fix was two standard steps — coalescing and reduction — both expressed in the DSL. The DSL reaches the same hardware integer-dot instruction and, with standard coalescing, *beats* the hand-written code.

³Throughput and latency are session benchmarks on evo (AMD Strix Halo, gfx1151; RADV/Vulkan and ROCm 7.13). The bit-exactness each accompanies is enforced by the committed tests of the oracle paper ([5]).

A migration that improved performance (add_rmsnorm). Two hand-written Vulkan kernels were drained on `main`: the DSL block-per-row `rms_norm` (270 GB/s) and `softmax` (268 GB/s) replaced naïve `.comp` shaders, each now the production kernel with its `.comp` deleted. The sharpest case is the fused residual-add + RMSNorm: the DSL `add_rmsnorm` twin ran **7.1× faster** than the incumbent shader ($166.5 \rightarrow 23.4$ ms at 4096^2 on RADV) *and* was bit-exact ($s = x + \text{res}$ byte-identical, y within 4.8×10^{-7}). The “uniformity” change was also a large speedup, because the incumbent was naïve and the DSL block kernel is coalesced. Table 2 collects the flipped cells.

Kernel	Backend	Outcome	Correctness
dp4a Q8 matvec	Vulkan (gfx1151)	196 GB/s, 118% of hand-tuned	bit-exact
<code>rms_norm</code>	Vulkan (gfx1151)	270 GB/s, <code>.comp</code> deleted	bit-exact
<code>softmax</code>	Vulkan (gfx1151)	268 GB/s, <code>.comp</code> deleted	bit-exact
<code>add_rmsnorm</code>	Vulkan (RADV)	$7.1 \times (166.5 \rightarrow 23.4 \text{ ms})$	bit-exact

Table 2: Cells flipped to the DSL on `main`. Every deletion cleared both gates: bit-exact against the oracle and at least as fast (here, faster). The engine is faster-or-equal at every step.

4 The discipline cuts both ways: the declines

A gate that never says no is not a gate. Where a hand-tuned kernel is *already optimal* — the Vulkan quant matvec, Metal’s `simd_sum` reductions, the fused `rope_norm` epilogue — a bit-exact DSL twin was built, measured, and *correctly not swapped*. A uniformity-only change with zero speedup and real regression risk is churn, not migration. Such a twin is kept as the portable *fallback* for backends that lack the tuned version, and the tuned peak stays the specialized fast path. “One source” is the goal, but the gate serves it only by also refusing the swaps that would trade speed for tidiness. We only ever add coverage and correctness.

5 An engineering note: opening a backend so it can flip

A cell cannot flip on a backend that does not build. Shipping the DSL on AMD required fixing two independent upstream defects *without* advancing the lowering engine past its newest published release (no version escape exists for either). First, the HIP FFI bindings are selected from `hipconfig --version`; ROCm 7.13 reports a patch past the newest shipped binding, so the upstream build emitted a feature with no bindings module and every symbol went undefined. The fix is a vendored [patch.crates-io] fork whose `build.rs` clamps an unknown-newer patch to the latest shipped binding — ABI-safe because AMD versions every changed symbol (R0600) and 7.13 still maps `hipGetDeviceProperties` to that revision. Second, the HIP server uses a `MultiStream` gated behind a `std-only` configuration the `hip/rocm` features do not pull, so the ROCm feature is declared to include `cubec1/std`. Both are recorded because a portable DSL lives or dies on the least-loved backend building cleanly — and only a backend that builds can have its cells verified and its hand-written kernels deleted.

6 The takeaway

Collapsing a Cartesian product of kernels is a migration problem, and migrations fail when they are faith-based or big-bang. Verify-then-delete makes both failure modes structurally impossible:

the CPU oracle supplies an objective correctness gate ([5]), the kernel-only benchmark supplies an objective performance gate, and the per-cell registry supplies incrementality. The result is an engine that is faster-or-equal at every intermediate state while its five hand-written kernel trees drain toward one source — and a boundary, honestly drawn, where one op ([6]) stays hand-written behind an island. The method is repeatable: point it at the next cell, run both gates, let the number decide.

References

- [1] Hanzo AI. *One Source, Every Backend: The hanzo-kernel DSL*. Paper 07 (umbrella).
- [2] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law*. Paper 07.1.
- [3] Hanzo AI. *The Schedule Is a Value: Comptime-Specialized Autotuning*. Paper 07.2.
- [4] Hanzo AI. *Intrinsic Islands: An Oracle-Anchored Escape Hatch*. Paper 07.3.
- [5] Hanzo AI. *One Oracle, Every Backend: The CPU Bit-Exactness Law*. Paper 07.4.
- [6] Hanzo AI. *When the Product Won't Collapse: An int8 Tensor-Core MMQ Negative Result*. Paper 07.5.
- [7] Hanzo AI. *Verify, Then Delete: Collapsing a Kernel Cartesian Product*. Paper 07.6 (this paper).